Ignytion io

IGNYTION IO · TECHNICAL LAB NOTE

UART protocol: From RTL to routed GDS

A beginner's walkthrough of a checked open-source reference build

Follow uart_core from its communication rules and Verilog **source** to a routed IHP SG13G2 layout. Worked calculations, measured digital traces and tool results are identified separately. This is an educational block, not a manufactured chip or a signoff-ready design.

STAGE	WHAT IT PRODUCED	RESULT
Simulation	Listed directed checks	PASS directed
Synthesis	543 mapped functional cells	PASS mapping
Routing	Routed DEF and extracted SPEF	PASS reference
Timing	TT and slow setup / hold	PASS reference
GDS	Decoded placement agreement	PASS export
Foundry DRC / LVS	No accepted result	UNPERFORMED

CONTENTS

What is in this document

Start with the wire-level rules, then follow the same circuit through the tools. Each section has a specific evidence boundary.

READING MAP

SECTION	PAGE
Start here	p. 03
Chip design in ten minutes	p. 04
UART logic is not RS-232	p. 05
Send bit zero first	p. 06
Integer prescale and sample drift	p. 07
Backpressure ends at the byte buffer	p. 08
Qualify START, then sample centres	p. 09
Framing and overrun are different faults	p. 10
Two different full-duplex bytes	p. 11
Lint: read code before running it	p. 12
Exhaustive bytes at one prescale	p. 13
Synthesis: a description becomes gates	p. 14
Floorplan and placement	p. 15
Clock tree and routing	p. 16
Two measured reference corners	p. 17
Count the right objects	p. 18
DRC and LVS: what is still missing	p. 19
Everything at a glance	p. 20
Run it yourself in WSL	p. 21
Sources, ownership and remaining work	p. 22
Design and physical glossary	p. 23–24

SECTION 1 · START HERE

Start here

A protocol is an agreement about how connected devices exchange information: which wire carries each bit, when to sample it, and how to recognise a complete transfer.

WHAT WE BUILT AND WHAT IT PROVES

A Verilog description represents circuits, not software instructions running one after another. Registers remember values; combinational logic calculates the next values. A **clock** advances the registered state. The next pages first explain this protocol at its wires, then connect those rules to the actual reference RTL.

The implemented unit is uart_core in hw/protocols/uart. It is a fixed **reference** block, not the Jane Street programmable combined emulator or a Tiny Tapeout submission top. No new competition architecture, instruction set, pin map or authority decision is implied by these documents.

All 256 bytes are tested at prescale 4 only. The 2..7 sweep uses fixed TX 0x96 and RX 0x69. No parity or hardware flow control. Later pages give the exact stimulus and the implementation evidence rather than using a broad protocol-compliant label. A successful mapped netlist, routed DEF and decoded **GDS** establish concrete implementation/export results, not readiness to manufacture.

The reading path starts with chip-design vocabulary, then develops the protocol, calculations, local interface and failure cases. Separate sections explain lint, simulation, synthesis, placement, routing, timing and **GDS**. The last two pages define design and physical terms. No prior ASIC-flow experience is assumed.

EVIDENCE VOCABULARY

Label	Meaning in this document
Worked example	Explicit assumptions; independently calculated
Measured trace	Positive digital regression VCD, where present
Directed PASS	Listed assertions and negative controls
Routed / decoded GDS	Physical export and structural agreement
Signoff	Not performed or claimed

SECTION 2 · CHIP DESIGN

Chip design in ten minutes

Each stage makes the same circuit more concrete. The folders below are the actual unit layout, not a copied APB command list.

CHIP DESIGN

RTL states what hardware should do. A simulator checks selected input sequences before any silicon exists. Synthesis replaces that description with gates from a process library. Physical tools choose positions and metal routes. Timing analysis checks the resulting delays; stream-out writes the mask-layout geometry.

THE PIPELINE, STAGE BY STAGE

FOLDER	WHAT HAPPENS HERE
00_lint	Read code for risky patterns; retained warnings
01_rtl	Pinned source and the local wrapper
02_sim	Independent stimulus, comparisons and fatal controls
03_verification	Supplemental checks; not a universal UVM PASS
04_synthesis	Map functional logic to SG13G2 cells
05_dft	Manufacturing test work remains unperformed
06_floorplanning	Die outline, rows, power and pin locations
07_placement	Choose legal positions for mapped cells
09_place_and_route	Clock tree, routing and SPEF extraction
08_sta	Read post-route TT and slow timing reports
10_gds	Stream-out and independent placement decoder
11_physical_verification	Foundry DRC / LVS remain unperformed

SECTION 3 · ELECTRICAL BOUNDARY

UART logic is not RS-232

The asynchronous framing convention does not specify the board-level voltage interface.

ELECTRICAL BOUNDARY

The local rxd and txd pins carry logic-level signals. RS-232 uses different voltage levels and polarity and cannot safely connect directly to ordinary digital pins. Full duplex uses independent wires in each direction. Flow-control pins such as RTS/CTS are optional system features absent from this wrapper.

A UART is a framing and timing function, not a promise about connector voltages. Never connect an RS-232 voltage directly to a low-voltage ASIC input. An external transceiver can convert levels and polarity. RS-485 uses differential signalling and may require driver-enable timing for a shared half-duplex link. Those electrical functions are not in uart_core.

UART tells us where a byte begins and ends. An application still needs to define how many bytes make a message and what the message means.

SEPARATE LOGIC FROM TRANSPORT

Element	Meaning
txd / rxd	Digital core-level serial signals
3.3 V / 5 V compatibility	Requires compatible pads or translation
RS-232	Requires an appropriate level-shifting transceiver
USB-to-UART bridge	Separate USB function; not supplied by this core

SECTION 4 · FRAME RULES

Send bit zero first

The fixed wrapper format is 8N1: START, eight data bits and one high STOP.

FRAME RULES

8N1 means eight data bits, no parity and one stop bit. It consumes ten bit periods per byte. There is no runtime parity setting.

Writing a number on paper usually puts the most significant bit on the left, while UART sends D0 first. The editable table shows temporal order.

For binary UART signalling, baud equals serial bit rate here. It does not equal useful **payload** bits per second because framing consumes two intervals per byte. The 32-byte calculation excludes idle gaps, scheduling delays and packet headers. If an application sends each value as ASCII digits, its useful measurement rate can be lower still. These are arithmetic examples for 8N1, not a measured link benchmark.

For 32 **payload** bytes in 8N1, the minimum is 32 x 10 = 320 wire bits. At 115200 baud this takes 320 / 115200 = 2.778 ms before gaps. The useful byte ceiling is 115200 / 10 = 11,520 bytes/s, even though the serial bit rate is 115200 bit/s.

WORKED BYTE 0X53

Field	Serial value
START	0
b0..b7	1 1 0 0 1 0 1 0
STOP	1
115200 baud useful byte ceiling	11,520 bytes/s before gaps

SECTION 5 · TIMING CALCULATION

Integer prescale and sample drift

The prescaler controls eight core-clock groups per data-bit interval.

TIMING CALCULATION

Nominal baud = fclk / (8P). At 50 MHz and a desired 115200 baud, the ideal integer divisor is 50,000,000 / (8 x 115200) = 54.253. Choosing P54 gives 115740.74 baud, +0.4694 percent error; P55 gives 113636.36 baud, -1.3573 percent. These are calculated examples. The full byte-space test runs at P4, and the separate fixed-byte sweep covers P2 through P7.

This simplified drift example is a timing model, not a universal two-percent acceptance rule. The receiver re-synchronizes on a new **START**, but not on every data bit. Long runs without data transitions still need correct local timing. The ideal half-bit margin is reduced by start detection, synchronisation, clock error and line distortion. Check a specific receiver data sheet or implementation before declaring an allowed total clock mismatch.

An ideal receiver checks **START** near its half-bit point, then samples data at 1.5, 2.5 through 8.5 bit intervals from the detected start and STOP at 9.5. At the P4 regression setting a bit is 0.64 us; nominal data-centre offsets are therefore 0.96, 1.60 through 5.44 us. The actual local RX counter waits half a bit to confirm START, then full-bit intervals for later samples. Its input synchroniser and detection delay must also be accounted for.

CALCULATED 50 MHZ EXAMPLES

Prescale	Baud	Error vs 115200
54	115740.74	+0.4694%
55	113636.36	-1.3573%
4, actual regression	1,562,500	Not a 115200-baud test

SECTION 6 · RTL INTERFACE

Backpressure ends at the byte buffer

Ready-valid can stop a local producer. It cannot pause bits already arriving on rxd.

RTL INTERFACE

Ready-valid allows a producer to hold a request until accepted. The selected transmitter does not provide an unbounded queue. Inspect upstream RTL to understand when it can accept another byte.

The **source** has a one-byte output register rather than an arbitrarily deep FIFO. Backpressure protects a byte only until another frame completes. A larger FIFO or external flow control would be another design. The current test exercises a limited ready stall and an isolated overrun; it does not promise lossless indefinite stalls.

The two-stage chain reduces propagation of metastability into the logic, but it cannot mathematically guarantee no failures. Its delay affects the **start** detection and must be considered with the baud interval. The marker guides implementation tools; it does not itself prove CDC correctness. The physical flow still needs proper placement and system integration checks before a reliability claim. A separate counter stage in upstream uart_rx samples this synchronised signal.

SOURCE / COMMANDS

```
(* async_reg = "true" *) reg [1:0] rx_sync;
always @(posedge clk)
if (rst) rx_sync <= 2'b11;
else rx_sync <= {rx_sync[0],rxd};
uart controller (
```

hw/protocols/uart/01_rtl/uart_core.v:24

SECTION 7 · CONTROL SEQUENCE

Qualify START, then sample centres

The receiver rejects short starts before committing to a full frame.

CONTROL SEQUENCE

When the initial candidate **START** does not remain low until the confirmation point, the receiver clears its bit counter and timer. If the line is still low, it advances to the data phase and loads a full-bit wait. This is a specific digital false-start check, not proof of immunity to every glitch width or analogue noise pattern.

The table corresponds to the counter-based uart_rx.v control sequence. A pending timer prevents a new sample until the proper interval expires. At count ten the receiver confirms **START**; at nine through two it shifts in the eight data bits; at one it evaluates STOP. It returns to zero after the frame. START detection initialises a half-bit delay and later data stages initialize full-bit delays. Output buffering and error flags are separate from this serial phase counter.

The data register includes a high bit for the **STOP** phase alongside the payload. txd becomes low for START while busy becomes high. Later timer expirations shift data bits into the output. No parity generation or alternative word length is enabled by this wrapper.

SOURCE-DERIVED RECEIVE SEQUENCE

bit_cnt	Phase	Action when timer expires
0	Idle	Observe low candidate START
10	Confirm START	Continue only if line stays low
9 through 2	Data b0 through b7	Shift sampled RX into data register
1	STOP	Accept data or flag framing error
0 again	Finish	Return to idle detection

SECTION 8 · FAILURE ANALYSIS

Framing and overrun are different faults

A low STOP is a wire-framing failure; a full output buffer is a consumer-capacity failure.

FAILURE ANALYSIS

A framing error checks the expected **STOP** level; it is not a checksum across the data. A wrong data bit with a valid STOP may go undetected by an 8N1 receiver. Sustained low after malformed STOP can resemble another START in this core, producing later recovery activity. The test drains that recovery before isolating the overrun check.

This **source** branch belongs to the STOP-sampling stage. A valid STOP writes the newly assembled data register and marks it available. The prior valid state becomes the overrun flag. This means the error reports loss/overwrite risk; it does not automatically queue another byte.

Synchronous **reset** requires a running clock to take effect. Reset polarity differs from the default I2C and SPI wrappers. Declaration initializers in upstream code are visible lint warnings. The actual reset path and mapped flops still require inspection.

FAULT AND CONSEQUENCE

Condition	Expected interpretation
STOP sampled low	frame_error
Completed byte with pending output	overrun / data-loss risk
Long low recovery stimulus	May resemble a new START
Data changed with valid STOP	May require application checksum to detect

SECTION 9 · MEASURED TRACE

Two different full-duplex bytes

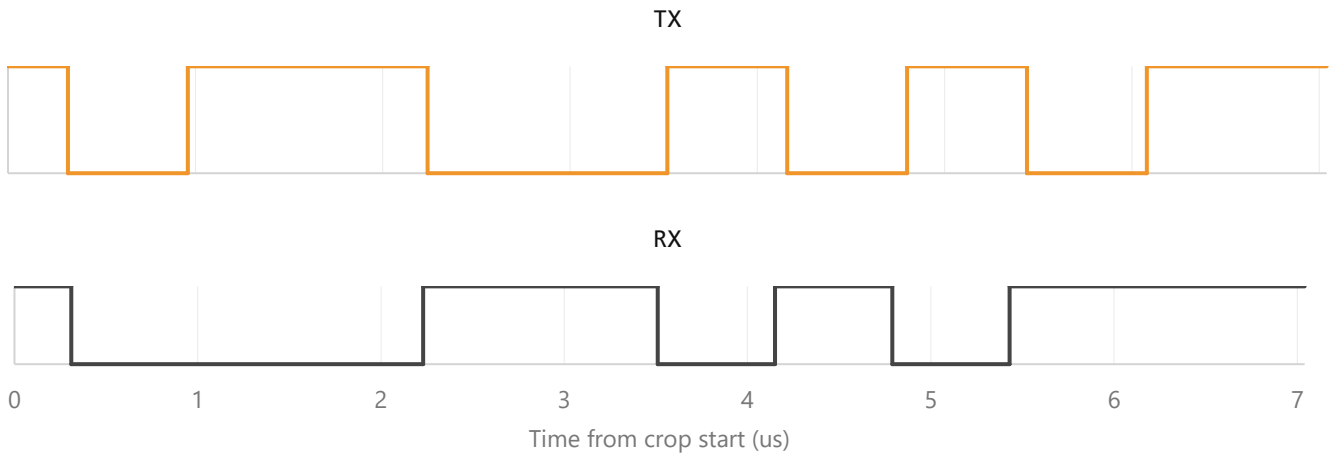
The waveform contains TX 0x53 and independently driven RX 0xAC.

MEASURED TRACE

At the actual 50 MHz simulation **clock**, prescale 4 produces 32 core clocks per bit, or 0.64 us. An 8N1 frame occupies ten bit intervals and therefore 6.4 us, before any idle gaps. Sampling TX at the eight data centres reconstructs 53; sampling the separately aligned RX centres reconstructs AC. Both STOP samples are high.

The received byte is the complement of the transmitted byte, so the plot demonstrates independent stimulus rather than TX-to-RX loopback. This is a fast directed digital regression at 1.5625 Mbaud, not a measured 115200-baud board capture. The illustrative prescale-54 configuration elsewhere is a calculation, not one of the tested sweep points.

MEASURED POSITIVE REGRESSION



Logic HIGH=1, LOW=0. Digital simulation, not analogue compliance; source hash retained.

SECTION 10 · LINT

Lint: read code before running it

Verilator checks legal but suspicious code patterns without exercising transactions. A warning count is a review starting point, not proof of correctness.

LINT

The retained report contains 20 warning messages and 0 error messages. Categories present: PROCASSINIT, WIDTHEXPAND. These figures count report message headers; repeated warnings are not necessarily independent defects. The report is hashed in this document evidence manifest.

The invocation deliberately uses -Wno-fatal so warnings remain visible while lint execution can complete. A successful command therefore does not mean every warning was resolved. Width changes, incomplete cases, **reset** use and unconnected ports need review against the wrapper contract and directed tests.

Lint cannot show that a receive byte is correct, that a transaction finishes, or that an external electrical interface is safe. Those questions require a checking testbench, implementation evidence and separate board or signoff work.

RETAINED WARNING REVIEW

MEASURE	VALUE
Warning message headers	20
Error message headers	0
Clean warning-free RTL?	Not claimed
Protocol behaviour proved by lint?	No

SOURCE / COMMANDS

```
make lint
# inspect, do not just count
# 00_lint/lint_report.txt
```

00_lint/Makefile and run_flow.sh: retained Verilator report

SECTION 11 · SIMULATION AND VERIFICATION

Exhaustive bytes at one prescale

A testbench is code that drives the circuit and judges its outputs. These directed checks use independent expected data and deliberately failing controls.

SIMULATION AND VERIFICATION

The full-duplex run tests all 256 byte values at prescale 4. A separate sweep uses prescales 2 through 7 with fixed TX 96 and RX 69. Combining those descriptions into all bytes at all prescalers would overstate the evidence. Neither test establishes arbitrary baud mismatch tolerance.

Directed checks include receive backpressure, malformed **STOP** and framing error, receive-buffer overrun, reset abort and a successful later transfer. The malformed-STOP source can remain low long enough to resemble another START. The bench drains the resulting recovery word before isolating the subsequent overrun check.

An injected wrong receive-byte expectation must fail fatally. The wrapper remains fixed to **8N1** with no parity, CTS/RTS or alternative word length. The two-stage receive synchroniser reduces metastability propagation, but these digital tests are not a CDC reliability or reset signoff result.

COVERAGE MATRIX

Case	Actual stimulus
Full byte space	256 values at prescale 4
Divider sweep	2..7; TX 96 / RX 69
Consumer faults	Stall, overrun, malformed STOP
Not tested	Parity, other word formats, arbitrary drift

> **A checker must be able to fail**
>
> Completion alone is insufficient. Read the successful log and the expected fatal rejection together; neither is a protocol-compliance certificate.

SECTION 12 · SYNTHESIS

Synthesis: a description becomes gates

A netlist is a list of logic cells and their connections. Yosys translates the RTL into cells from the IHP SG13G2 library.

SYNTHESIS

A process design kit, or PDK, contains the process-specific files used by implementation tools. This **reference** uses IHP SG13G2, not the SKY130 library used by the golden APB primer. Standard cells are predesigned gates and storage elements with characterised timing and physical views.

The **source** is compiled and checked under the unit workspace before synthesis. Yosys maps the functional logic into the pinned IHP SG13G2 standard-cell library. The current netlist contains 543 functional cells, excluding $scopeinfo metadata. The structural checker rejects remaining generic logic, latches or unresolved instances.

The mapped count excludes $scopeinfo metadata. It is not the count of every placed instance after **clock** buffers, repair cells and fillers are added. It also does not mean that the original protocol has been independently proven for all possible transactions.

MAPPING RESULT

MEASURE	CURRENT RESULT
Top module	uart_core
Mapped functional cells	543
Technology	IHP SG13G2
Unresolved generic logic / latches	Rejected by structural checker

SOURCE / COMMANDS

```
make synth
node verify_build.mjs synth
```

04_synthesis/synth.tcl; verify_build.mjs synthesis checks

SECTION 13 · PHYSICAL IMPLEMENTATION

Floorplan and placement

The floorplan gives the circuit a physical outline. Placement chooses where its cells sit inside that outline.

PHYSICAL IMPLEMENTATION

The die outline for this standalone block is 300 x 300 um. The floorplan establishes rows and **reference** power/pin geometry. The placer must fit cells legally without overlaps and with room for routing. This is a block-level educational layout, not a complete padframe or package design.

Mapped functional logic has 543 cells. After physical additions and fillers, the routed DEF contains 16,512 placed standard-cell instances. A filler occupies empty row sites; it is not another functional logic gate. That distinction explains why the two counts differ so much.

Utilisation is the fraction of core area occupied by placed cell area. It is not simply functional-cell count divided by die size, and it changes between stages. This note does not substitute APB utilisation, row count or I/O count for unavailable protocol-specific measurements.

ACTUAL PHYSICAL REFERENCE

MEASURE	VALUE
Die outline	300 x 300 um
Mapped functional cells	543
Placed standard-cell instances	16,512
Physical inputs	06_floorplanning; 07_placement
Placement agreement	DEF compared with decoded GDS

SECTION 14 · CLOCK AND WIRES

Clock tree and routing

Clock-tree synthesis distributes clocks to registers. Routing draws the metal connections between placed cells.

CLOCK AND WIRES

A register samples its input at a **clock** edge. Clock buffers distribute that edge across the block; the difference in arrival times is called skew. The reference is constrained at 50 MHz / 20 ns.

OpenROAD completes the staged physical flow. The **reference** die is 300 x 300 um. DEF records routed design geometry and cell placement; SPEF captures extracted parasitics used by the timing analysis. The scripts retain stage-specific output so the result can be investigated instead of relying on a final banner.

Real interconnect has resistance and capacitance, called parasitics. Extraction writes these in **SPEF** for post-route timing. The retained router-rule DRC file is empty. That checks the router view of its work; it is not a full foundry-deck check of the exported polygons.

READ THE PHYSICAL EVIDENCE

ARTIFACT	QUESTION IT ANSWERS
09_place_and_route/route.def	Where are cells and routed connections?
09_place_and_route/route.spef	Which parasitics were extracted?
Router-rule DRC report	Any reported routing-rule violations?
08_sta/timing_report.txt	What does extracted reference timing report?

> **Two checks can both be called DRC**
>
> The empty router report is accepted routing evidence. Foundry DRC is a separate unperformed obligation; one result cannot stand in for the other.

SECTION 15 · REFERENCE TIMING

Two measured reference corners

Static timing analysis adds cell and wire delays without simulating input patterns. Setup asks whether data arrives early enough; hold asks whether it stays stable long enough.

REFERENCE TIMING

The physical **clock** constraint is 50 MHz / 20 ns. Post-route timing uses the extracted design rather than an ideal pre-route wire estimate. The report at each corner must be inspected for worst setup and hold slack and for VIOLATED markers. The reported values are not a timing guarantee under unstated board, voltage or temperature conditions.

Current TT setup/hold slack is +15.75 / +0.17 ns. The slow-corner values are +14.45 / +0.24 ns. Both reports contain no VIOLATED marker. These numbers are taken directly from the local report files at document build time; the evidence manifest retains their hashes.

I/O delays are **reference** budgets, not a contract with an external target, transceiver or PHY. Bus synchronisation, reset recovery/removal, all additional corners and modes, signal integrity and board timing need their own analysis. Positive slack in these two reports is therefore called reference timing closure, not foundry or system signoff.

LIVE EXTRACTED SLACK, NS

Corner	Worst setup	Worst hold
TT	15.75	0.17
Slow	14.45	0.24
Status	PASS reference reports	No VIOLATED markers

> **Positive slack has a stated boundary**
>
> Slack is remaining timing margin. These values apply to the listed clocks, I/O budgets and two library corners; they are not a board-timing or manufacturing signoff guarantee.

SECTION 16 · GDS STREAM-OUT

Count the right objects

GDS stores layout polygons and hierarchical cell references. Stream-out combines routed geometry with the real standard-cell layouts from the PDK.

GDS STREAM-OUT

The **GDS** file contains 1,284,780 bytes and 45 structures. The independent decoder finds 16,512 placed standard-cell instances including fillers. That placement count is intentionally much larger than the 543 mapped functional cells: filler instances occupy unused placement sites and structures are unique hierarchy definitions.

The decoder compares exported cell identities, coordinates and orientations against the routed DEF. A deliberately altered placement/export expectation fails, including under Python optimisation, so acceptance is not implemented with removable assert-only checks. Raw file SHA256 is a run-specific fingerprint because stream-out timestamps can change.

The OpenROAD router-rule DRC report is empty. It is not a foundry-deck DRC result. LVS, antenna, scan/DFT, power integrity, padframe, tap/endcap strategy and manufacturing signoff remain unperformed. No empty routing report is promoted into a DRC-clean tapeout claim.

LIVE EXPORT MEASUREMENTS

Quantity	Current value
Mapped functional cells	543
GDS bytes	1,284,780
GDS structures	45
Placed standard-cell instances	16,512
Die	300 x 300 um

SECTION 17 · UNPERFORMED SIGNOFF

DRC and LVS: what is still missing

Design rule checking asks whether geometry follows the process rules. Layout versus schematic asks whether the extracted layout circuit matches the intended netlist.

UNPERFORMED SIGNOFF

Foundry DRC requires the applicable, qualified rule deck and an inspected report on the final polygons. Typical checks include minimum widths, spacing and enclosure. No foundry-deck DRC result has been accepted for this protocol **reference**. The router report is narrower evidence.

LVS requires extracting a circuit from the layout and comparing devices and connectivity with the intended circuit. A structural **GDS** decoder checking cell references, coordinates and orientations does not perform that extraction or connectivity comparison. No accepted LVS match is available.

Antenna, scan/DFT, power integrity, padframe, tap/endcap strategy and manufacturing release remain unperformed. The golden APB primer reports its own separate checks; those results do not transfer to SG13G2 or to this design.

SEPARATE ACCEPTED EXPORT FROM MISSING SIGNOFF

CHECK	THIS REFERENCE
Router-rule report	Empty; narrower routing evidence
Decoded GDS placement	PASS structural/export agreement
Foundry-deck DRC	UNPERFORMED
Extracted-layout LVS	UNPERFORMED
Manufacturing readiness	Not claimed

> **A GDS file is not tapeout approval**
>
> This package is suitable for studying and reproducing a block implementation. Do not hand it off as a signoff-clean manufacturing design.

SECTION 18 · RESULTS SUMMARY

Everything at a glance

Read each result together with the question it actually answers. The same PASS label never implies every protocol or manufacturing obligation is closed.

RESULTS SUMMARY

The directed functional result and the physical export result are both useful, but they prove different things. The physical build uses the fixed wrapper configuration and **reference** constraints. Additional modes, external peripherals, arbitrary clock relationships and board operation require their own evidence.

Important protocol boundary: All 256 bytes are tested at prescale 4 only. The 2..7 sweep uses fixed TX 0x96 and RX 0x69. No parity or hardware flow control.

FULL REFERENCE FLOW

STAGE	TOOL / EVIDENCE	STATUS
Lint	Verilator; 20 retained warnings	Review
Simulation	Independent directed expectations	PASS directed
Negative controls	Deliberate fatal mismatches	PASS rejection
Synthesis	Yosys; 543 mapped cells	PASS mapping
Floorplan / placement	OpenROAD; 300 x 300 um	PASS reference
Routing / extraction	DEF and SPEF retained	PASS reference
Timing	TT and slow setup / hold	PASS reference
GDS	KLayout; 1,284,780 bytes	PASS export
Foundry DRC / LVS	No accepted report	UNPERFORMED
Full-chip / board	No accepted integration proof	UNPERFORMED

SECTION 19 · RUN IT YOURSELF

Run it yourself in WSL

Use the pinned source, the unit tool lock and the external PDK revision together.

RUN IT YOURSELF

The proven entry point is a WSL login Bash shell. igny init prepares the packaging runtime. Unsetting CRUCIBLE_ENV after initialisation allows the unit crucible.toml workspace selection to control tool resolution rather than an inherited environment override. This is the observed workflow, not a change to Crucible internals.

Each unit has its existing environment and lock. The common tool set includes Icarus, Verilator, Yosys, OpenROAD and klayout-stream. The separate PDK checkout is IHP SG13G2 revision 22f43352dd8219f9007eb659e422e0d5fe28c5fb. A tool lock does not pin that external PDK by itself; IHP_PDK_ROOT must point to the matching installation.

The documented machine could not use the pip klayout package as the expected stream-out executable; igny tool install klayout-stream supplies the proven converter. make lint sim checks functional behaviour, make flow executes the staged implementation, and node verify_build.mjs all inspects artifacts. Review normal and intentionally failing logs before accepting the result.

Re-running a simulation can replace its VCD and invalidate an earlier document trace hash. Retain a document evidence manifest or regenerate the measured figures after the positive run. Do not silently preserve an old chart while claiming it is the newly generated waveform.

SOURCE / COMMANDS

```
igny init
unset CRUCIBLE_ENV
cd hw/protocols/uart
igny workspace sync
igny tool install klayout-stream
make lint sim
make flow
node verify_build.mjs all
```

Run after entering the repository in WSL login Bash; PDK required.

SECTION 20 · SOURCES AND REMAINING WORK

Sources, ownership and remaining work

The pinned revision identifies the baseline; local integration changes are described separately.

SOURCES AND REMAINING WORK

Upstream: https://github.com/alexforencich/verilog-uart
Pinned commit: 1b867e53af738e4a8bc7c839ca2f1c07f40382dc
Licence: MIT, Alex Forencich, 2014-2017. Original **source** notices remain in the RTL. The document is authored by Ignytion; this does not transfer ownership of upstream code or remove its licence obligations.

Local modifications, directed test stimulus and **source** boundaries are recorded in hw/protocols/uart/README.md and the repository append-only .context.md. The first three protocol lessons have separate editable 16:9 student decks; this A4 technical note is a reading and evidence document, not a replacement for their instructor notes.

Primary reading: Microchip EUSART/AUSART framing **reference**, plus the pinned verilog-uart counters and handshake.
Primary framing reference:
https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/8bit-pic/peripherals/eusart-ausart/

Open work remains explicit: All 256 bytes are tested at prescale 4 only. The 2..7 sweep uses fixed TX 0x96 and RX 0x69. No parity or hardware flow control. Full-chip integration, manufacturing checks and board validation are not closed by the standalone **reference** flow. The reader can reproduce the listed evidence without assuming those larger obligations have already been completed.

SECTION 21 · DESIGN VOCABULARY

Glossary: design and protocol

Definitions for reading the protocol rules and the checked RTL. Physical implementation terms follow on the next page.

DESIGN AND VERIFICATION

TERM	WHAT IT MEANS
Protocol	Rules for exchanging information between devices
RTL	Registers and the logic moving values between them
Verilog	A language used to describe hardware
FSM	A circuit progressing through defined states
Lint	Static source checks; no transaction execution
Testbench	Code that supplies inputs and checks outputs
Coverage	Which defined situations were exercised
Negative control	An intentionally wrong case the checker must reject
VCD	Digital signal-change trace from a simulation
Handshake	Signals agreeing when data is accepted
8N1	Eight data bits, no parity, one STOP bit
Prescale	Integer number of core clocks per eighth-bit interval
Frame error	STOP sample was not high
Overrun	New completed receive data met an occupied output buffer

SECTION 22 · PHYSICAL VOCABULARY

Glossary: implementation and signoff

These definitions help distinguish a successfully exported reference layout from a manufacturing-ready design.

PHYSICAL IMPLEMENTATION

TERM	WHAT IT MEANS
Synthesis	Translate RTL into a technology-cell circuit
Netlist	Cells and the connections between them
Standard cell	Predesigned logic/storage cell in a process library
PDK	Process design kit: process-specific implementation files
SG13G2	IHP process used by these reference builds
Floorplan	Die/core outline, rows, power and pin geometry
Placement	Choose legal positions for cells
Filler	Physical row-site cell, not functional logic
CTS / skew	Clock distribution / difference in arrival times
Routing / via	Metal connections / links between metal layers
Parasitics / SPEF	Wire resistance/capacitance / extracted data format
Corner	A characterised process, voltage and temperature case
STA / slack	Static timing analysis / remaining timing margin
DEF / GDS	Placement/routing exchange / polygon hierarchy
DRC	Check geometry against applicable process rules
LVS	Compare the extracted layout circuit with its netlist
Signoff	Accepted applicable release checks; unperformed here

© 2026 Ignytion IO · ignytion.io